

The background of the slide features a large, faint watermark of the Universidad Mayor de San Andrés (UMSA) logo. It is an inverted triangle with a red border. Inside, the top half is white with the letters 'UMSS' in blue. The bottom half is split into two red triangles meeting at a central point, where a small grey geometric logo is located.

UMSS

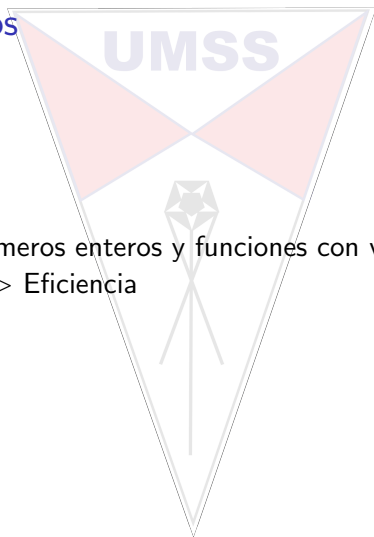
Teoría de números

Leticia Blanco

Departamento de Informática - Sistemas
UMSS

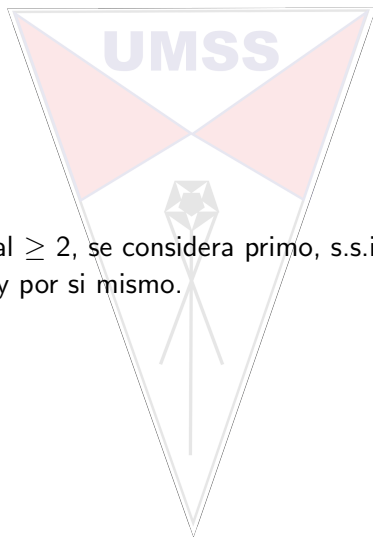
30 de agosto de 2024

Teoría de números



Estudio de los números enteros y funciones con valores enteros.
Importancia — > Eficiencia

Números primos



Un numero natural ≥ 2 , se considera primo, s.s.i. es divisible por 1 y por si mismo.

Algoritmo 1

Intervalo de divisores excepto 1 y n , $[2, n - 1]$

```
1  boolean esPrimo1(int n){
2      boolean primo;
3      int div;
4      primo = false;
5      if(n > 1){
6          primo = true;
7          div = 2;
8          while(div <= n-1 && primo){
9              primo = n % div != 0;
10             div++;
11         }
12     }
13     return primo;
14 }
```

$O(n)$

Algoritmo 2

Se reduce el intervalo de divisores $[2, \sqrt{n}]$

```

1  boolean esPrimo2(int n){
2      boolean primo;
3      int div;
4      int lim;
5      primo = false;
6      if(n > 1){
7          primo = true;
8          div = 2;
9          lim = (int)Math.sqrt(n);
10         while(div <= lim && primo){
11             primo = n % div != 0;
12             div++;
13         }
14     }
15     return primo;
16 }
```

Algoritmo 3

Se reduce el intervalo de divisores impares $[3, \sqrt{n}]$, el unico primo par es 2

```

1  boolean esPrimo3(int n){
2      boolean primo;
3      int div;
4      int lim;
5      if(n == 2)
6          primo = true;
7      else if(n == 1 || n % 2 == 0)
8          primo = false;
9      else{
10         primo = true;
11         div = 3;
12         while(div <= Math.sqrt(n) && primo){
13             primo = n % div != 0;
14             div += 2;
15         }
16     }
17     return primo;

```

Algoritmo 4

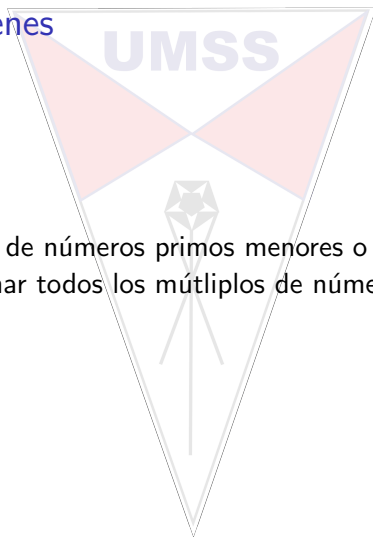
Se reduce el intervalo de divisores primos $[2, \sqrt{n}]$,

```

1  boolean esPrimo4(int n){
2      boolean primo;
3      int ind;
4      int[] divs = primos((int)Math.sqrt(n));
5      primo = false;
6      if(n >= 2){
7          ind = 0;
8          primo = true;
9          while(ind < divs.length && primo){
10             primo = n % divs[ind] != 0;
11             ind++;
12         }
13     }
14     return primo;
15 }
```

$$O(\sqrt{n} / \ln \sqrt{n})$$

Criba de Eratostenes



Encuentra la lista de números primos menores o iguales a n .
Consiste en eliminar todos los múltiplos de números primos.

Criba de Eratostenes

Primos ≤ 59

*	*	2	3	4	5	6	7	8	9
10	11	12	13	14	15	16	17	18	19
20	21	22	23	24	25	26	27	28	29
30	31	32	33	34	35	36	37	38	39
40	41	42	43	44	45	46	47	48	49
50	51	52	53	54	55	56	57	58	59

$2 \leq \sqrt{59}$

*	*	2	3	*	5	*	7	*	9
*	11	*	13	*	15	*	17	*	19
*	21	*	23	*	25	*	27	*	29
*	31	*	33	*	35	*	37	*	39
*	41	*	43	*	45	*	47	*	49
*	51	*	53	*	55	*	57	*	59

Criba de Eratostenes

$$3 \leq \sqrt{59}$$

*	*	2	3	*	5	*	7	*	*
*	11	*	13	*	*	*	17	*	19
*	*	*	23	*	25	*	*	*	29
*	31	*	*	*	35	*	37	*	*
*	41	*	43	*	*	*	47	*	49
*	*	*	53	*	55	*	*	*	59

$$4 \leq \sqrt{59} ; 5 \leq \sqrt{59}$$

*	*	2	3	*	5	*	7	*	*
*	11	*	13	*	*	*	17	*	19
*	*	*	23	*	*	*	*	*	29
*	31	*	*	*	*	*	37	*	*
*	41	*	43	*	*	*	47	*	49
*	*	*	53	*	*	*	*	*	59

Criba de Eratostenes

$$6 \leq \sqrt{59}; 7 \leq \sqrt{59}$$

*	*	2	3	*	5	*	7	*	*
*	11	*	13	*	*	*	17	*	19
*	*	*	23	*	*	*	*	*	29
*	31	*	*	*	*	*	37	*	*
*	41	*	43	*	*	*	47	*	*
*	*	*	53	*	*	*	*	*	59

por lo tanto:

2	3	5	7	11	13	17	19	23	29
31	37	41	43	47	49	53	59		

La cantidad de operaciones es:

$$n * (1/2 + 1/3 + 1/5 + 1/7 + \dots, 1/\text{ultimoPrimo})$$

Donde ultimo primo es el ultimo del rango acotado por $\sqrt{n}$

Por lo que este algoritmo tiene una complejidad:

$$O(n * \log \log n)$$

Numeros compuestos

Son aquellos que no son primos.

Se pueden representar de manera única como la multiplicación de sus factores primos.

Teorema fundamental de la aritmética:

Todo numero entero se construye como la multiplicación de algún conjunto de números primos.

$$n = 4896 = 2 * 2 * 2 * 2 * 2 * 3 * 3 * 17 = 2^5 * 3^2 * 17$$

factorización
en potencias
de primos

Factores primos

```
1 List<Integer> factores(int n){
2     List<Integer> facts;
3     int[] primos = primos((int) Math.sqrt(n));
4     facts = new ArrayList<Integer>();
5     for(int i = 0; i < primos.length; i++){
6         while(n % primos[i] == 0){
7             n = n / primos[i];
8             facts.add(primos[i]);
9         }
10    }
11    return facts;
12 }
```

$$O(\sqrt{n}/\ln \sqrt{n})$$

Similar a este es contar los factores primos

Divisores

Cuantos divisores hay?

```
1  int nroDivisores(int n){
2      int cantidad;
3      cantidad = 2; // 1 y n
4      for(int i = 2; i <= n/2; i++){
5          if(n % i == 0)
6              cantidad++;
7      }
8      return cantidad;
9  }
```

$O(n/2)$

Divisores

Otra opción es usando los factores primos, si:

$$n = a^i * b^j * c^k *n^z$$

El numero de divisores de n es:

$$(i + 1) * (j + 1) * (k + 1) * ... * (z + 1)$$

Por ejemplo:

$$60 = 1, 2, 3, 4, 5, 6, 10, 12, 15, 20, 30, 60$$

$$60 = 2^2 * 3^1 * 5^1$$

El numero de divisores es:

$$(2 + 1) * (1 + 1) * (1 + 1) = 12$$

$$O(\sqrt{n} / \ln \sqrt{n})$$

Suma de divisores

Cuál es la suma de los divisores de n ?

```
1  int nroDivisores(int n){
2      int suma;
3      suma = 1+n;
4      for(int i = 2; i <= n/2; i++){
5          if(n % i == 0)
6              suma += i;
7      }
8      return suma;
9  }
```

$O(n/2)$

Suma de divisores

Otra opción es usando los factores primos, si:

$$n = a^i * b^j * c^k *n^z$$

La suma de divisores de n es:

$$\frac{a^{i+1}-1}{a-1} * \frac{b^{j+1}-1}{b-1} * \frac{c^{k+1}-1}{c-1} * ... * \frac{n^{z+1}-1}{n-1}$$

Por ejemplo:

$$60 = 2^2 * 3^1 * 5^1$$

La suma de los divisores es:

$$\frac{2^{2+1}-1}{2-1} * \frac{3^{1+1}-1}{3-1} * \frac{5^{1+1}-1}{5-1} = \frac{2^3-1}{1} * \frac{3^2-1}{2} * \frac{5^2-1}{4} = 7 * 4 * 6 = 168$$

$$O(\sqrt{n} / \ln \sqrt{n})$$

Máximo común divisor - mcd

Algoritmo de Euclides:

```
1  int mcd(int a, int b){  
2      return b == 0? a: mcd(b, a % b);  
3  }
```

$O(\log_{10} n)$ donde $n = \min(a, b)$

Mínimo común múltiplo - mcm

Algoritmo de Euclidiano:

```
1  int mcm(int a, int b){  
2      return a * (b / mcd(b, a));  
3  }
```

$O(\log_{10} n)$ donde $n = \min(a, b)$

Aritmética Modular

UMSS

Entender la aritmética modular es importante:

Definition

x modulo N es el residuo de dividir x entre N

$$x = q * N + r; 0 \leq r < N$$

$\therefore x$ modulo N es igual a r

Congruencia Modular

Se dice que x e y son congruentes modulares si:

Definition

$$x \equiv y \pmod{N} \Leftrightarrow N \text{ divide a } (x - y)$$

Example

$$-9 \equiv -6 \pmod{3}$$

$$-19 \equiv -7 \pmod{6}$$

Esto es muy útil, porque bajo algunas condiciones da lo mismo trabajar con -19 que con -7

Sustitución

Definition

$$x \equiv x' \pmod{N} \wedge y \equiv y' \pmod{N} \rightarrow x + y \equiv x' + y' \pmod{N} \wedge xy \equiv x'y' \pmod{N}$$

Example

$$-9 \equiv -6 \pmod{3}$$

$$-19 \equiv -7 \pmod{3}$$

$$\rightarrow -9 + (-19) \equiv -6 + (-7) \pmod{3}$$

Sustitución

Example

Hay 25 episodios de 3 horas, que se empezaron a ver a la medianoche, a que hora del día se terminará de ver la serie?

$$(25 * 3) \bmod 24$$

$\wedge$

$25 \equiv 1 \pmod{24}$, entonces puedo usar 1 en vez de 25 y queda:

$1 * 3 = 3 \bmod 24$, lo que significa que terminará de ver a las 3:00 a.m.

Sustitución

Definition

Asociativa $x + (y + z) \equiv (x + y) + z \pmod{N}$

Conmutatividad $xy \equiv yx \pmod{N}$

Distributiva $x(y + z) \equiv xy + yz \pmod{N}$

Example

A que es equivalente $2^{345} \pmod{31}$?

Sustitución

Definition

Asociativa $x + (y + z) \equiv (x + y) + z \pmod{N}$

Conmutatividad $xy \equiv yx \pmod{N}$

Distributiva $x(y + z) \equiv xy + yz \pmod{N}$

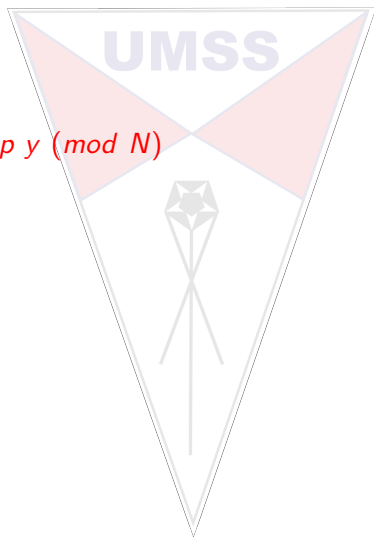
Example

A qué es equivalente $2^{345} \pmod{31}$?

$$2^{345} \equiv (2^5)^{69} \equiv 32^{69} \equiv 1^{69} \equiv 1 \pmod{31}$$

Costos

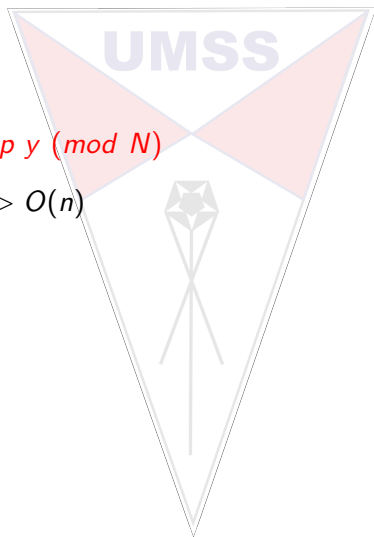
Para la forma $x \text{ op } y \pmod N$



Costos

Para la forma $x \text{ op } y \pmod N$

Suma modular $\Rightarrow O(n)$

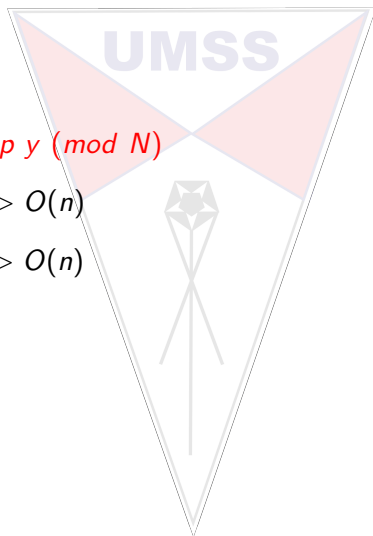


Costos

Para la forma $x \text{ op } y \pmod N$

Suma modular $\Rightarrow O(n)$

Resta modular $\Rightarrow O(n)$



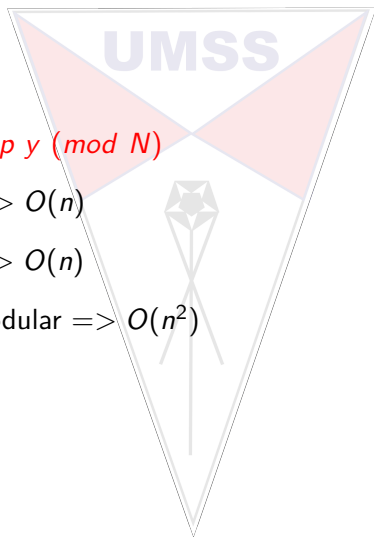
Costos

Para la forma $x \text{ op } y \pmod N$

Suma modular $\Rightarrow O(n)$

Resta modular $\Rightarrow O(n)$

Multiplicación modular $\Rightarrow O(n^2)$



Costos

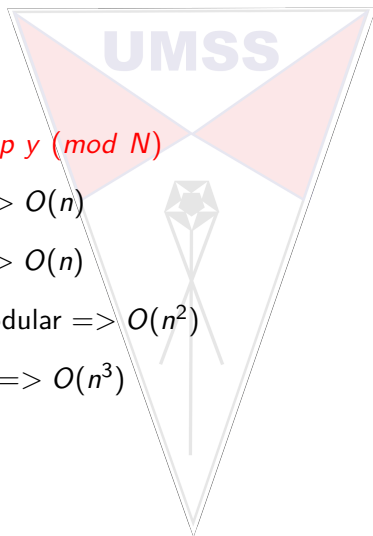
Para la forma $x \text{ op } y \pmod N$

Suma modular $\Rightarrow O(n)$

Resta modular $\Rightarrow O(n)$

Multiplicación modular $\Rightarrow O(n^2)$

División modular $\Rightarrow O(n^3)$



Costos

Para la forma $x \text{ op } y \pmod{N}$

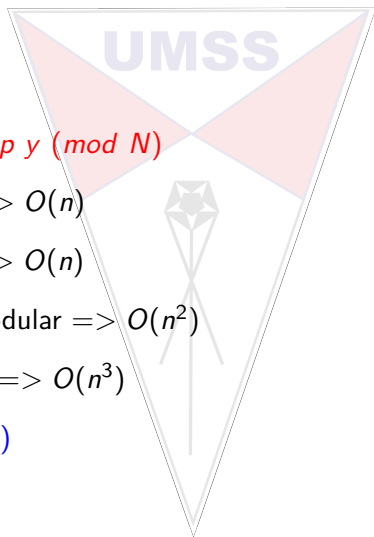
Suma modular $\Rightarrow O(n)$

Resta modular $\Rightarrow O(n)$

Multiplicación modular $\Rightarrow O(n^2)$

División modular $\Rightarrow O(n^3)$

Donde $n = \log(N)$

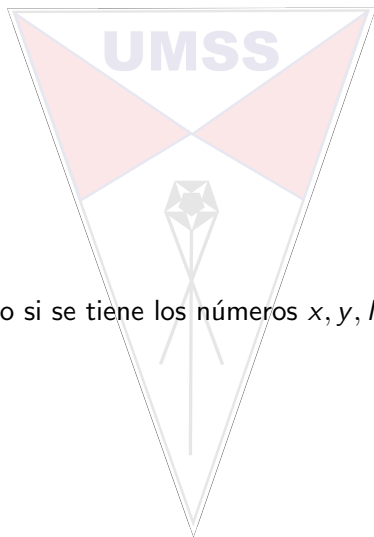


Exponenciación

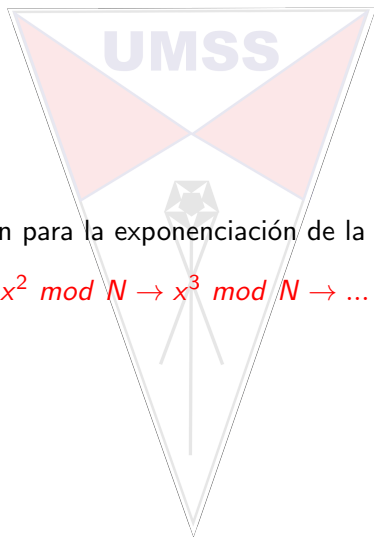
Definition

$$x^y \bmod N$$

Se vuelve complejo si se tiene los números x, y, N grandes.



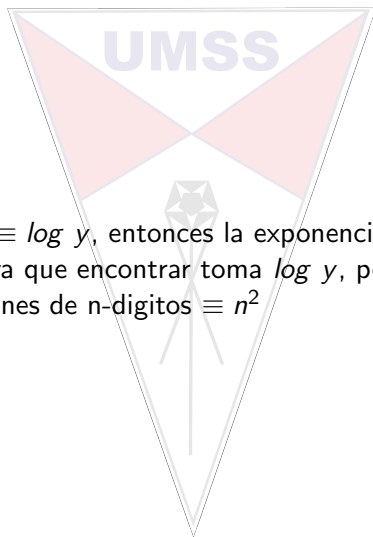
Exponenciación



Existe una relación para la exponenciación de la siguiente manera:

$$x \bmod N \rightarrow x^2 \bmod N \rightarrow x^3 \bmod N \rightarrow \dots \rightarrow x^y \bmod N$$

Exponenciación



Si se tiene que $n \equiv \log y$, entonces la exponenciación se puede hacer en $O(n^3)$, ya que encontrar $\log y$ toma $\log y$, pero cada vez se hace multiplicaciones de n -dígitos $\equiv n^2$

División modular

Definition

x es el inverso multiplicativo de a mod N si,
 $ax \equiv 1 \pmod{N}$



mcd de Euclides extendido

Además de encontrar el mcd, el extendido encuentra los coeficientes x , y de la identidad de Bézout (lema).

Definition

Si d es el $\text{mcd}(a,b)$:

$$d = a*x + b*y$$

Aquí bastaría con encontrar los coeficientes x e y

mcd de Euclides extendido

```
1  int x, y;
2  int euclidesExt(int a, int b){
3      int x1, y1, q, t;
4      x1 = 0; y1 = 1; y = 0; x = 1;
5      while(b != 0){
6          q = a/b;
7          t = b; b = a % b; a = t;
8          t = x1; x1 = x - q * x1; x = t;
9          t = y1; y1 = y - q * y1; y = t;
10     }
11 }
```